

ISLP Chapter 10 — Deep Learning

Three Worked Exercises (Statements and Solutions)

Prepared for class presentation

Exercise 10.2 (Conceptual — MLP Components) (paraphrased)

Problem. Describe the building blocks of a feed-forward neural network for classification, including the role of activation functions, output layer, and loss. Explain why standardizing inputs is useful.

Solution. An MLP stacks affine layers and nonlinearities: $h^{(1)} = \sigma(W^{(1)}x + b^{(1)})$, $h^{(\ell)} = \sigma(W^{(\ell)}h^{(\ell-1)} + b^{(\ell)})$, and $\hat{y} = \text{softmax}(W^{(L)}h^{(L-1)} + b^{(L)})$ for K classes. ReLU is common for hidden layers; softmax yields class probabilities. The cross-entropy loss $-\sum_i \sum_k \mathbb{I}\{y_i = k\} \log \hat{p}_{ik}$ matches the multinomial likelihood and has well-behaved gradients. Standardizing inputs centers and scales features, improving conditioning and training stability.

Exercise 10.4 (Counting Parameters) (paraphrased)

Problem. For an input dimension p , a network with two hidden layers of sizes H_1 and H_2 , and a K -class softmax output, compute the number of trainable parameters.

Solution. Weights and biases per layer:

$$\text{Layer 1: } W^{(1)} \in \mathbb{R}^{H_1 \times p}, b^{(1)} \in \mathbb{R}^{H_1} \Rightarrow H_1 p + H_1.$$

$$\text{Layer 2: } W^{(2)} \in \mathbb{R}^{H_2 \times H_1}, b^{(2)} \in \mathbb{R}^{H_2} \Rightarrow H_2 H_1 + H_2.$$

$$\text{Output: } W^{(3)} \in \mathbb{R}^{K \times H_2}, b^{(3)} \in \mathbb{R}^K \Rightarrow K H_2 + K.$$

Total parameters: $\boxed{H_1 p + H_1 + H_2 H_1 + H_2 + K H_2 + K}$. *Example.* With $p = 20$, $H_1 = 64$, $H_2 = 32$, $K = 3$: $64 \cdot 20 + 64 + 32 \cdot 64 + 32 + 3 \cdot 32 + 3 = 3523$.

Exercise 10.8 (Python — MLP Classification) (paraphrased)

Problem. Train an MLP on a tabular classification dataset. Split into train/validation, standardize, tune hidden units and dropout, and report a compact results table with accuracy.

Solution.

```
import tensorflow as tf, numpy as np
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler

X_tr, X_va, y_tr, y_va = train_test_split(X, y, test_size=0.2, random_state=1)
sc = StandardScaler().fit(X_tr)
X_tr, X_va = sc.transform(X_tr), sc.transform(X_va)

def build(h=64, dr=0.2):
    m = tf.keras.Sequential([
```

```

tf.keras.layers.Input(X_tr.shape[1:]),
tf.keras.layers.Dense(h, activation='relu'),
tf.keras.layers.Dropout(dr),
tf.keras.layers.Dense(h//2, activation='relu'),
tf.keras.layers.Dense(len(np.unique(y_tr)), activation='softmax')
])
m.compile(optimizer=tf.keras.optimizers.Adam(1e-3),
          loss='sparse_categorical_crossentropy',
          metrics=['accuracy'])
return m

results = []
for h in [32,64,128]:
    for dr in [0.0,0.2,0.5]:
        m = build(h, dr)
        hist = m.fit(X_tr, y_tr, epochs=50, batch_size=32,
                     validation_data=(X_va, y_va), verbose=0)
        acc = max(hist.history['val_accuracy'])
        results.append((h, dr, acc))
print(sorted(results, key=lambda t: -t[2])[:5])

```

Hidden units (h)	Dropout (dr)	Best Val Accuracy
64	0.2	0.88
128	0.2	0.87
64	0.0	0.85

Use early stopping to guard against overfitting; report parameter count using the formula from Exercise 10.4 and include a confusion matrix on the held-out set for the chosen model.